

Feedback Report Homework Assignment 1

Mursal Stella Mohammadi

October 4, 2026

Correction Feedback integrated

The feedback was:

1. You are using the NFS to read the files. Please provide a version without that.
2. processes accumulate on the first node you target
3. you provide hw details only of the frontend node of the cluster if i understand correctly.”

1. Feedback point

In the optMPT.c I already had incooperated the requirement that there shall be no usage of NFS. I have replaced the file distribution part from mainMPI.c with the one from optMPI.c.

...

```
// load encrypted file only on rank 0
uint32_t ciphertext_length = 0;
uint32_t sha_length = 0;

if (rank == 0)
{
    ciphertext_length = file_load(enc_path, ciphertext);
    if (ciphertext_length <= 0)
    {
        printf("Failed to load .enc file\n");
        err = 1;
    }
    sha_length = file_load(sha_path, plaintext_checksum);
    if (sha_length != SHA512_DIGEST_LENGTH)
    {
        printf("Failed to load .sha file\n");
    }
}
```

```

        err = 1;
    }
}

// Broadcast the err to the other ranks
MPI_Bcast(&err, 1, MPI_INT, 0, MPLCOMM_WORLD);
if (err)
{
    free(ciphertext);
    free(plaintext);
    MPI_Finalize();
    return 0;
}

// Broadcast the ciphertext, length and plaintext to the other ranks
MPI_Bcast(&ciphertext_length, 1, MPI_UINT32_T, 0, MPLCOMM_WORLD);
MPI_Bcast(&sha_length, 1, MPI_UINT32_T, 0, MPLCOMM_WORLD);
MPI_Bcast(ciphertext, ciphertext_length, MPI_UINT8_T, 0, MPLCOMM_WORLD);
MPI_Bcast(plaintext_checksum, SHA512_DIGEST_LENGTH, MPI_UINT8_T,
0, MPLCOMM_WORLD);
...
}

```

2. Feedback point

I have now fixed that with including a .sh file that enables updating *refresh_hostfile.sh* which is used as in the following.

```

chmod +x refresh_hostfile.sh //one time
./refresh_hostfile.sh

```

The following alteration of the command allows me to use many more slots which improves the speed very much. An example command to find a password

```

mpirun -np 200 --hostfile hostfile --map-by node --bind-to core ./optMPI 5
./files/test3/test3.enc ./files/test3/test3.sha512

```

3. Feedback point

In the section of my report I have written the Hardware and system environment of the frontend node only. So for the current hostfile I have added a script calling and saving lscpu on the hostfile nodes (*gather.sh*). Then a second script to reduce the lscpu output to Model name, Thread(s) per core, Core(s) per socket, Socket(s), NUMA node(s) (). And counting the amount of each of the node occurrences. This leaves me to the following information about the computing nodes.

```
chmod +x gather.sh
./gather.sh hostfile reports
```

```
chmod +x summarize.sh
./summarize.sh reports
```

Which leads to the following output:

Total nodes: 82

```
— 31 node(s) —
Model name: Intel(R) Core(TM) i7-7700 CPU @ 3.60GHz
Thread(s) per core: 1
Core(s) per socket: 4
Socket(s): 1
NUMA node(s): 1
```

```
— 24 node(s) —
Model name: Intel(R) Core(TM) i7-9700 CPU @ 3.00GHz
Thread(s) per core: 1
Core(s) per socket: 8
Socket(s): 1
NUMA node(s): 1
```

```
— 7 node(s) —
Model name: Intel(R) Xeon(R) Silver 4114 CPU @ 2.20GHz
Thread(s) per core: 1
Core(s) per socket: 10
Socket(s): 1
NUMA node(s): 1
```

...

The shared university SSH cluster is heterogeneous, querying all available 82 nodes in the hostfile with `lscpu` revealed 17 distinct hardware configurations. Nodes vary in CPU model (Intel Core i5/i7/i9 and Intel Xeon E5/Silver/Bronze/W series, 2.2–3.8 GHz), core count (4–20), socket count (1–2) and NUMA nodes (1–2).

Dominant types:

- 31× Intel Core i7-7700 (4 cores, 1 socket)
- 24× Intel Core i7-9700 (8 cores, 1 socket)
- 10× Intel Xeon Silver 4114 (10 or 20 cores, 1–2 sockets)

This leads us to the following types of **available** nodes: